Introduction to Machine Learning for Text Analysis with Python
Day 3 – Wednesday
»Supervised Machine Learning with transformers«

Rupert Kiddle
Sjoerd Stolwijk

r.t.kiddle@vu.nl, @rptkiddle
s.b.stolwijk@uu.nl

September 16, 2026

Today

Recap

Embeddings

- Encoding text in ML models
- Non-Contextual Embeddings
- Using word embeddings to improve models

Neural networks

- Contextual Embeddings

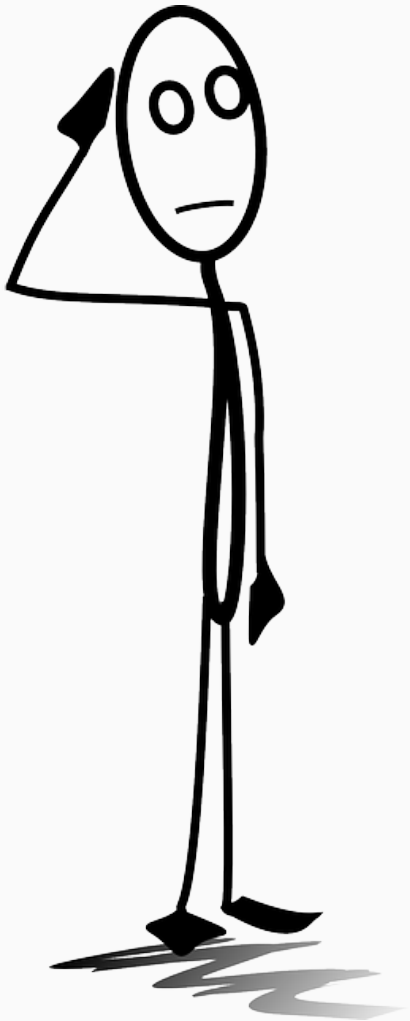
Transformer-based models

Transfer learning paradigm

- Do I need all this fancy stuff?

Ethical considerations

Exercise



Everything clear from this morning?

Recap

How Humans Learn vs. How Machines Learn

Human Learning

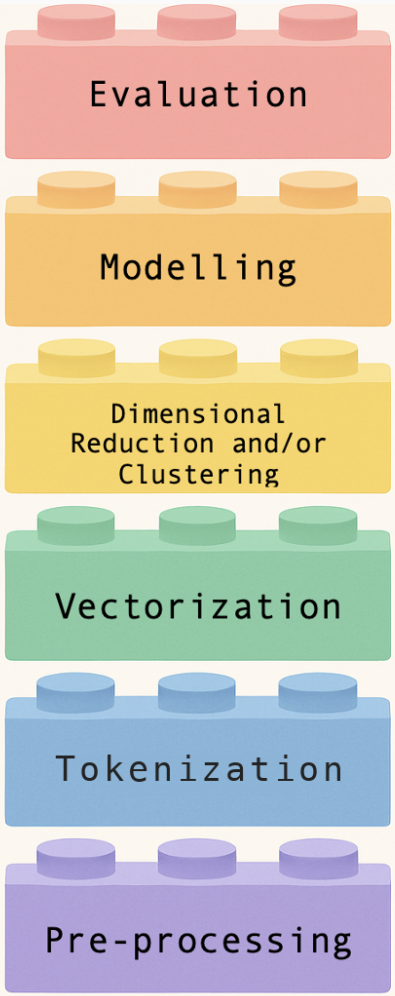
Example counting
(seeing many repetitions)

Pointing out
(explicit correction / feedback)

Explaining in other words
(paraphrase / abstraction)

Interpreting context
(discourse / world knowledge)

Lego pipeline



how to assess the output of your model

how to train a model to describe (UML) or to predict (SML)

how to group features *or* cases

how to turn text → numbers

how to split text into tokens

how to tidy up text data

Timeline of Key Embedding Models

- **Word2Vec** – 2014
  Introduced efficient neural word embeddings.

- **Transformers** – 2017
  Vaswani et al. introduced the Transformer architecture, enabling powerful sequence modeling.

- **BERT** – 2018
  Devlin et al. introduced BERT, a bidirectional transformer for language understanding.

- **ChatGPT-3.5** – 2022
  OpenAI released ChatGPT-3.5, a large language model for conversational AI.

Two big jumps

With respect to the basic Naïve Bayes classifier with a CountVectorizer we make two big jumps this afternoon:

- From BoW to contextual embeddings
- From linear prediction to neural networks

Disclaimer: We cannot give a full overview of the whole topic of deep learning and transformer models here – that's a whole (extensive) course in itself. But embeddings are closely related, so we will at least get our feet wet a bit.

Embeddings

Our BOW approach until now

Representing a document by word frequency counts

Result of preprocessing and vectorizing:

0. `He took the dog for a walk to the dog playground`

$\Rightarrow$ `took dog walk dog playground`

$\Rightarrow$ `'took':1, 'dog': 2, walk: 1, playground: 1`

Consider these other sentences

1. He took the doberman for a walk to the dog playground
2. He took the cat for a walk to the dog playground
3. He killed the dog on his walk to the dog playground

The vectorized representations of these sentences have a "distance" (dissimilarity) of 1 each, but arguably, sentences 0 and 1 should be "closer" than others

Our BOW approach until now

Representing a document by word frequency counts

Result of preprocessing and vectorizing:

0. `He took the dog for a walk to the dog playground`

$\Rightarrow$ `took dog walk dog playground`

$\Rightarrow$ `'took':1, 'dog': 2, walk: 1, playground: 1`

Consider these other sentences

1. He took the doberman for a walk to the dog playground
2. He took the cat for a walk to the dog playground
3. He killed the dog on his walk to the dog playground

The vectorized representations of these sentences have a "distance" (dissimilarity) of 1 each, but arguably, sentences 0 and 1 should be "closer" than others

Our BOW approach until now

Representing a document by word frequency counts

Result of preprocessing and vectorizing:

0. `He took the dog for a walk to the dog playground`

$\Rightarrow$ `took dog walk dog playground`

$\Rightarrow$ `'took':1, 'dog': 2, walk: 1, playground: 1`

Consider these other sentences

1. He took the doberman for a walk to the dog playground
2. He took the cat for a walk to the dog playground
3. He killed the dog on his walk to the dog playground

The vectorized representations of these sentences have a "distance" (dissimilarity) of 1 each, but arguably, sentences 0 and 1 should be "closer" than others

Our BOW approach until now

Representing a document by word frequency counts

Result of preprocessing and vectorizing:

0. `He took the dog for a walk to the dog playground`

⇒ `took dog walk dog playground`

⇒ `'took':1, 'dog': 2, walk: 1, playground: 1`

Consider these other sentences

1. He took the doberman for a walk to the dog playground
2. He took the cat for a walk to the dog playground
3. He killed the dog on his walk to the dog playground

The vectorized representations of these sentences have a "distance" (dissimilarity) of 1 each, but arguably, sentences 0 and 1 should be "closer" than others

Our BOW approach until now

Representing a document by word frequency counts

Result of preprocessing and vectorizing:

0. `He took the dog for a walk to the dog playground`

⇒ `took dog walk dog playground`

⇒ `'took':1, 'dog': 2, walk: 1, playground: 1`

Consider these other sentences

1. He took the doberman for a walk to the dog playground
2. He took the cat for a walk to the dog playground
3. He killed the dog on his walk to the dog playground

The vectorized representations of these sentences have a "distance" (dissimilarity) of 1 each, but arguably, sentences 0 and 1 should be "closer" than others

Embeddings

Encoding text in ML models

Encoding so far: BoW approach

- Our vectorizers gave a random ID to each word
- Words are *discrete* and *independent* tokens.
- This is a rather naïve assumption, with two main disadvantages:
  1. high dimensionality of the tokens
  2. they do not incorporate real-world knowledge

Token	Index	One-hot vector
aargh	0	[1,0,0]
king	1	[0,1,0]
queen	2	[0,0,1]

Embeddings

Non-Contextual Embeddings

"...a word is characterized by the company it keeps..." (Firth, 1957)

Word embeddings . . .

- help capture the meaning of text
- are low-dimensional vector representations that capture semantic meaning
- for instance, 'dobermann' and 'bulldog' should be represented by vectors that are close to each other in space, while 'kill' and 'walk' should be far from each other.

From static to contextual embeddings

Transformers:

- Dynamically compute word representations based on the entire sentence.
- Each word's embedding can change depending on surrounding words.
- Attention mechanism allows the model to focus on relevant words anywhere in the input.

Training contextual word embeddings

- Language modelling is a set of techniques that aim to determine the probability of a given sequence of words occurring in a sentence.
- Large language Models: Language modelling applied to massive amounts of training data (e.g., Wikipedia, news archives, Reddit, etc.)
- Often referred to as 'pretrained' or 'foundation' models.

Training language models: based on unstructured text – in the absence of explicit labels

Dobermann

From Wikipedia, the free encyclopedia

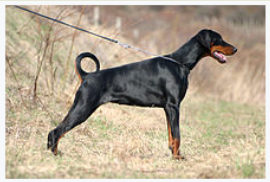
"Doberman" redirects here. For other uses, see Doberman (disambiguation).

This article **needs additional citations for verification**. Please help improve this article by adding citations to reliable sources. Unsourced material may be challenged and removed.
Find sources: "Dobermann" – news · newspapers · books · scholar · JSTOR
(March 2018) (Learn how and when to remove this template message)

The **Dobermann** (/ˈdoʊbərmən/; German pronunciation: [ˈdoːbɐman]), or **Doberman Pinscher** in the United States and Canada, is a medium-large breed of domestic dog that was originally developed around 1890 by Louis Dobermann, a tax collector from Germany.[2] The Dobermann has a long muzzle. It stands on its pads and is not usually heavy-footed. Ideally, they have an even and graceful gait. Traditionally, the ears are cropped and posted and the tail is docked. However, in some countries, these procedures are now illegal and it is often considered cruel and unnecessary. Dobermanns have markings on the chest, paws/legs, muzzle, above the eyes, and underneath the tail.

Dobermanns are known to be intelligent, alert, and

Dobermann



Other names	Doberman Pinscher
Common nicknames	Dobie, Doberman
Origin	Germany

Traits [hide]

Height	Dogs	68 to 72 cm (27 to 28 in)[1]
	Bitches	63 to 68 cm (25 to 27 in)[1]

Bulldog

From Wikipedia, the free encyclopedia

This article is about the English Bulldog. For other uses, see Bulldog (disambiguation).
See also: French Bulldog, American Bulldog, and Old English Bulldog

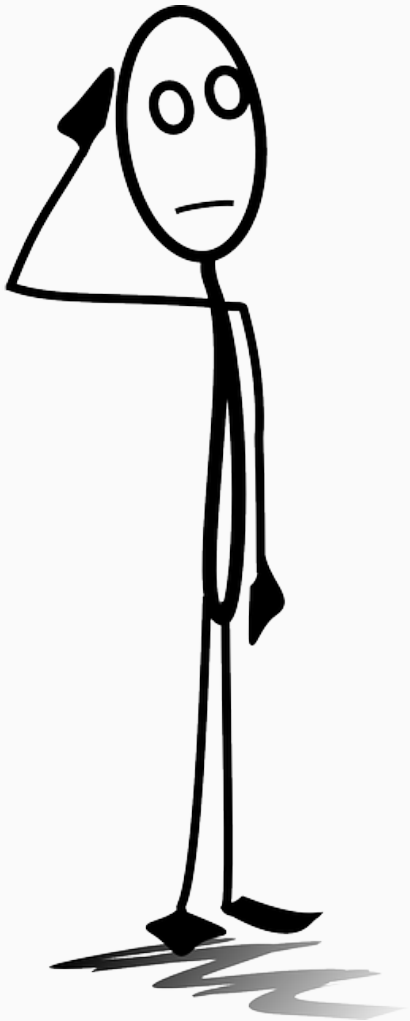
The **Bulldog** is a British breed of dog of mastiff type. It may also be known as the **English Bulldog** or **British Bulldog**. It is of medium size, a muscular, hefty dog with a wrinkled face and a distinctive pushed-in nose.[4] It is commonly kept as a companion dog; in 2013 it was in twelfth place on a list of the breeds most frequently registered worldwide.[5]

The Bulldog has a longstanding association with British culture; the BBC wrote: "to many the Bulldog is a national icon, symbolising pluck and determination".[6] During the Second World War, the Prime Minister Winston Churchill was likened to a Bulldog for his defiance of Nazi Germany.[7] The Bulldog Club (In England) was formed in 1878, and the Bulldog Club of America was formed in 1890.

Bulldog

Male English Bulldog

Other names	English Bulldog, British Bulldog
Origin	England[1]



What can we use word embeddings for?

Embeddings

Improving down-stream classification tasks

In supervised machine learning

- Modify CountVectorizer or TfidfVectorizer:
- For each document, we find the embedding of each word with reference to a pre-trained embedding model (e.g., trained on the whole Wikipedia)
- We then aggregate these embeddings (e.g., mean or sum)
- For each document, we now have a, for example, 300-dimensional instead of 10,000 dimesional vector.
- Use these vectors to predict the label.

In supervised machine learning

- Modify CountVectorizer or TfidfVectorizer:
- For each document, we find the embedding of each word with reference to a pre-trained embedding model (e.g., trained on the whole Wikipedia)
- We then aggregate these embeddings (e.g., mean or sum)
- For each document, we now have a, for example, 300-dimensional instead of 10,000 dimesional vector.
- Use these vectors to predict the label.

In supervised machine learning

- Modify CountVectorizer or TfidfVectorizer:
- For each document, we find the embedding of each word with reference to a pre-trained embedding model (e.g., trained on the whole Wikipedia)
- We then aggregate these embeddings (e.g., mean or sum)
- For each document, we now have a, for example, 300-dimensional instead of 10,000 dimesional vector.
- Use these vectors to predict the label.

In supervised machine learning

- Modify CountVectorizer or TfidfVectorizer:
- For each document, we find the embedding of each word with reference to a pre-trained embedding model (e.g., trained on the whole Wikipedia)
- We then aggregate these embeddings (e.g., mean or sum)
- For each document, we now have a, for example, 300-dimensional instead of 10,000 dimesional vector.
- Use these vectors to predict the label.

In supervised machine learning

- Modify CountVectorizer or TfidfVectorizer:
- For each document, we find the embedding of each word with reference to a pre-trained embedding model (e.g., trained on the whole Wikipedia)
- We then aggregate these embeddings (e.g., mean or sum)
- For each document, we now have a, for example, 300-dimensional instead of 10,000 dimesional vector.
- Use these vectors to predict the label.

In supervised machine learning

What does this mean?

- Our embedding model is larger, but our number of features is smaller
- We can use words in the prediction set *even if they are not in the training dataset* (as long as it is in the embedding model!)
- We can learn from similar training samples even if they do not use the same exact words
- But the process of running classifications via neural networks is more computationally demanding

In supervised machine learning

What does this mean?

- Our embedding model is larger, but our number of features is smaller
- We can use words in the prediction set *even if they are not in the training dataset* (as long as it is in the embedding model!)
- We can learn from similar training samples even if they do not use the same exact words
- But the process of running classifications via neural networks is more computationally demanding

In supervised machine learning

What does this mean?

- Our embedding model is larger, but our number of features is smaller
- We can use words in the prediction set *even if they are not in the training dataset* (as long as it is in the embedding model!)
- We can learn from similar training samples even if they do not use the same exact words
- But the process of running classifications via neural networks is more computationally demanding

In supervised machine learning

What does this mean?

- Our embedding model is larger, but our number of features is smaller
- We can use words in the prediction set *even if they are not in the training dataset* (as long as it is in the embedding model!)
- We can learn from similar training samples even if they do not use the same exact words
- But the process of running classifications via neural networks is more computationally demanding

Example Visualization

Ongoing work within the TWON project, visualization credit to Simon Münker, Trier University

Let's do an exercise :)

Neural networks

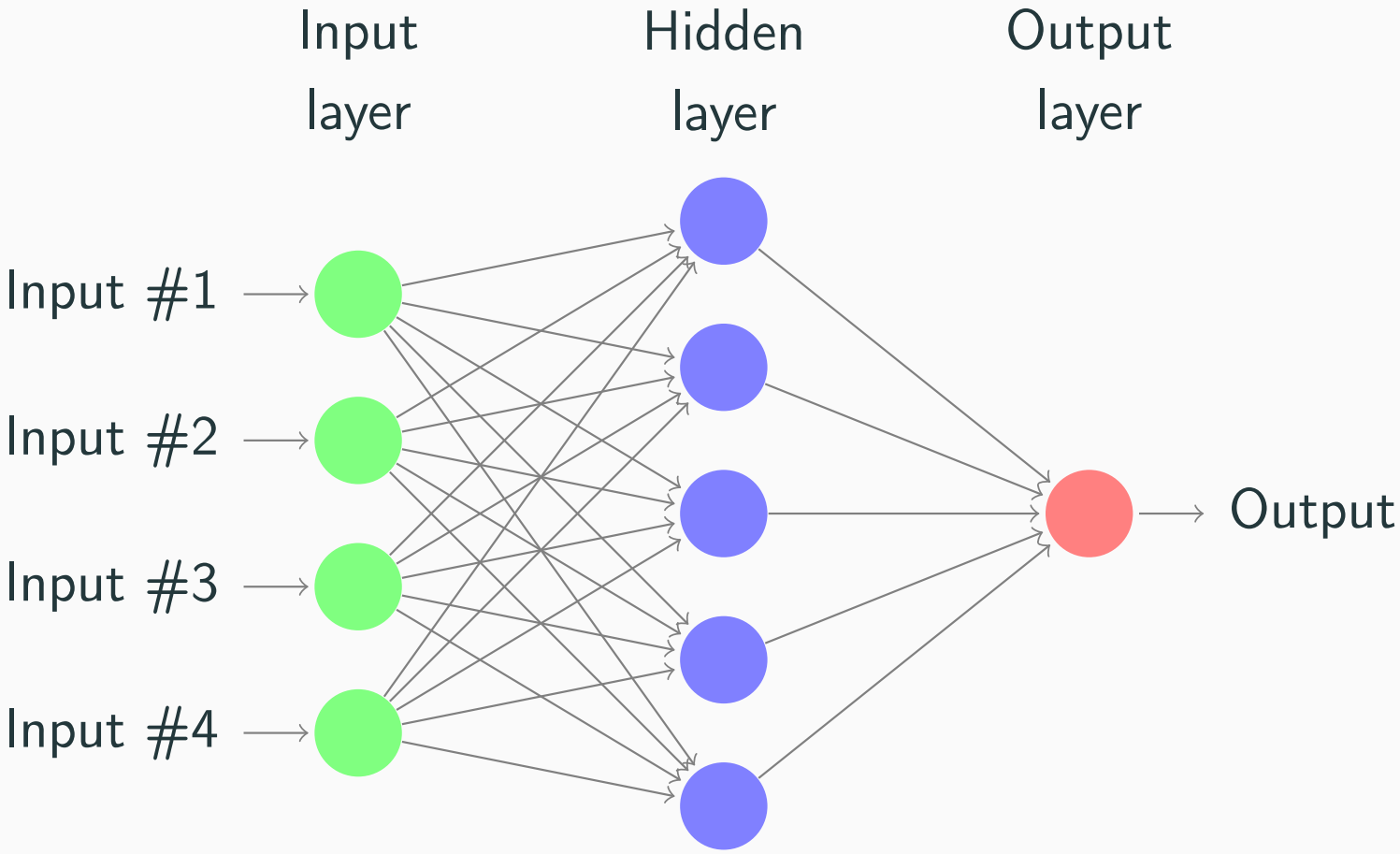
Recap: Two big jumps

With respect to the basic Naïve Bayes classifier with a CountVectorizer we make two big jumps this afternoon:

- From BoW to contextual embeddings
- From linear prediction to neural networks

Neural Networks

- In "classical" machine learning, we predict an outcome directly based on the input features
- In neural networks, we can have "hidden layers" that we predict
- These layers are not necessarily interpretable
- "Neurons" that "fire" based on an "activation function"



$\Rightarrow$ If we had multiple hidden layers in a row, we'd call it a *deep* network.

Why neural networks?

- learn hidden structures (e.g., embeddings (!))
- go beyond the idea that there is a direct relationship between occurrence of word X and label (or occurrence of pixel [R,G,B] and a label)
- images, machine translation — and more and more general NLP, sentiment analysis, etc.

Example of a comparatively easy introduction:
https://towardsdatascience.com/
neural-network-embeddings-explained-4d028e6f0526

Convolutional networks

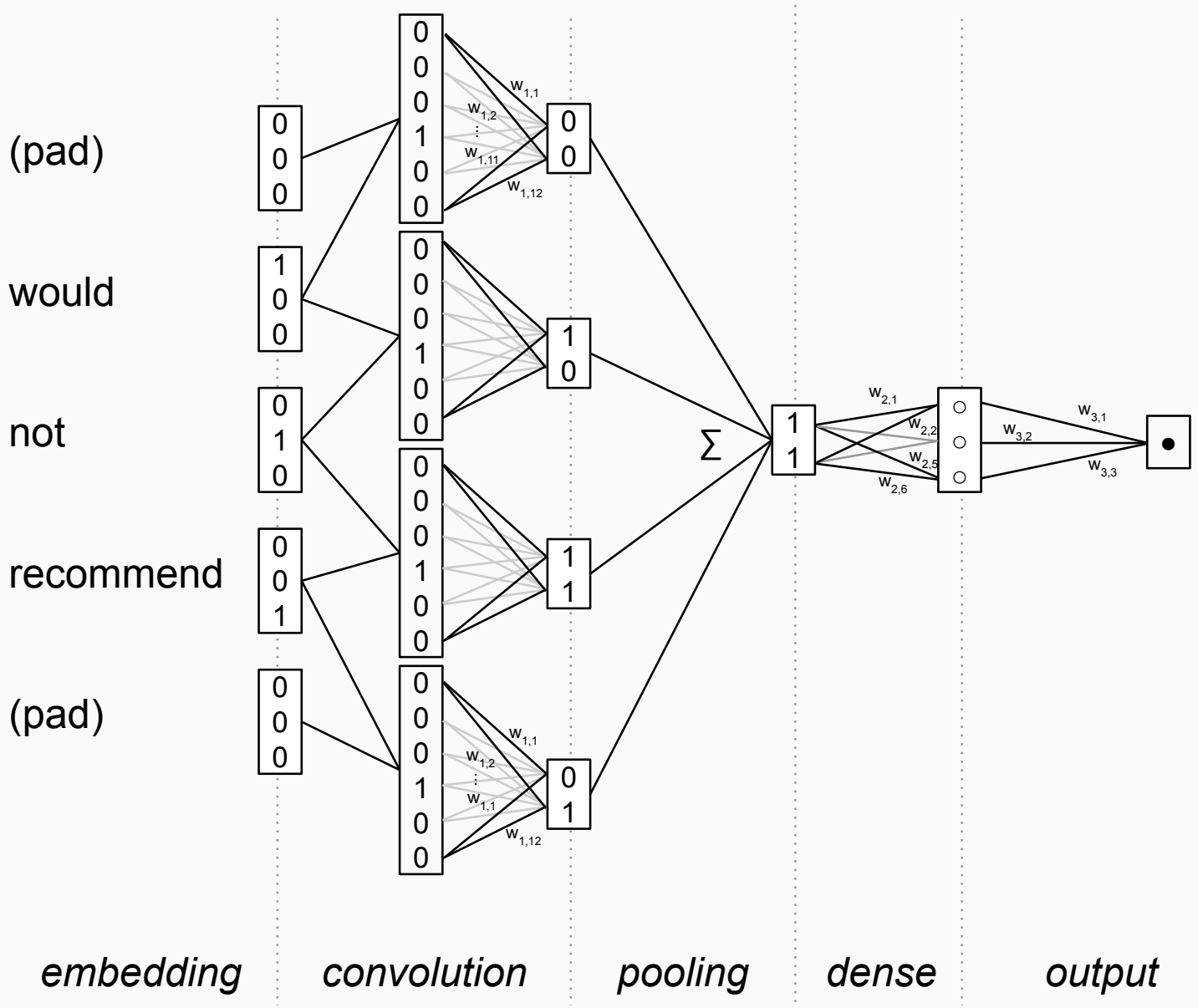
The problem with such a basic network: just as with classic SML, we still lose all information about order (the "not good" problem).

Therefore,

- We concatenate the vectors of neighboring words
- We apply some filter (essentially, we detect patterns)
- and then pool the results (e.g., taking the maximum)

This means that we now explicitly take into account *the temporal structure* of a sentence.

Convolutional networks



Neural networks

Contextual Embeddings

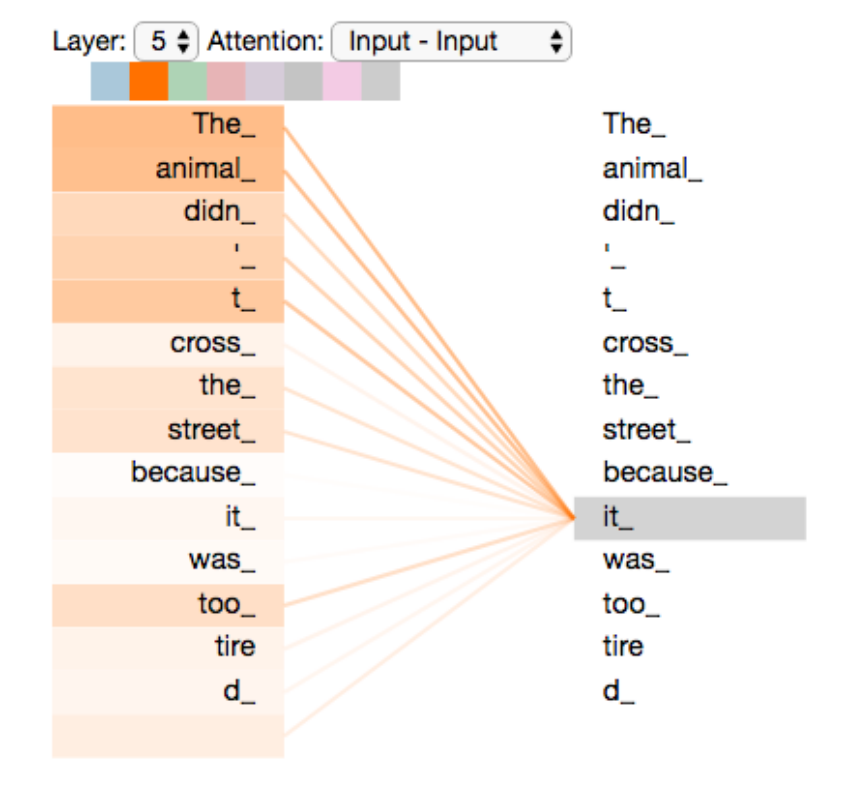
Enter: Contextual Word Embeddings

- *Transformers* create a new vector for each time a word is used in the dataset
- *Contextualized* vectors.
- *self-attention* mechanism is essential here: this is a manner to automatically decide which nearby words should influence a token's representation; the model *learns* which tokens *to attend to*

Transformer-based models

Attention Mechanism

"The animal didn't cross the street because it was too tired"



Source: Jay Alammar - "The Illustrated Transformer"

Transformer-based models

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2019)

- (Huge) pre-trained model (by, e.g., Google)
- Trained on very large amount of text and can use words in context.
- BERT and other transformer-based models are used for a range of tasks;
  1. Sentiment analysis (+/-)
  2. sequence-to-sequence predictions (e.g., translation)
  3. similarity and paraphrasing tasks
  4. natural language inference tasks (token prediction)

Transformer-based models

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2019)

- (Huge) pre-trained model (by, e.g., Google)
- Trained on very large amount of text and can use words in context.
- BERT and other transformer-based models are used for a range of tasks;
  1. Sentiment analysis (+/-)
  2. sequence-to-sequence predictions (e.g., translation)
  3. similarity and paraphrasing tasks
  4. natural language inference tasks (token prediction)

Transformer-based models

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2019)

- (Huge) pre-trained model (by, e.g., Google)
- Trained on very large amount of text and can use words in context.
- BERT and other transformer-based models are used for a range of tasks;
  1. Sentiment analysis (+/-)
  2. sequence-to-sequence predictions (e.g., translation)
  3. similarity and paraphrasing tasks
  4. natural language inference tasks (token prediction)

Transformer-based models

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2019)

- (Huge) pre-trained model (by, e.g., Google)
- Trained on very large amount of text and can use words in context.
- BERT and other transformer-based models are used for a range of tasks;
  1. Sentiment analysis (+/-)
  2. sequence-to-sequence predictions (e.g., translation)
  3. similarity and paraphrasing tasks
  4. natural language inference tasks (token prediction)

Transformer-based models

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2019)

- (Huge) pre-trained model (by, e.g., Google)
- Trained on very large amount of text and can use words in context.
- BERT and other transformer-based models are used for a range of tasks;
  1. Sentiment analysis (+/-)
  2. sequence-to-sequence predictions (e.g., translation)
  3. similarity and paraphrasing tasks
  4. natural language inference tasks (token prediction)

Transformer-based models

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2019)

- (Huge) pre-trained model (by, e.g., Google)
- Trained on very large amount of text and can use words in context.
- BERT and other transformer-based models are used for a range of tasks;
  1. Sentiment analysis (+/-)
  2. sequence-to-sequence predictions (e.g., translation)
  3. similarity and paraphrasing tasks
  4. natural language inference tasks (token prediction)

Transformer-based models

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2019)

- (Huge) pre-trained model (by, e.g., Google)
- Trained on very large amount of text and can use words in context.
- BERT and other transformer-based models are used for a range of tasks;
  1. Sentiment analysis (+/-)
  2. sequence-to-sequence predictions (e.g., translation)
  3. similarity and paraphrasing tasks
  4. natural language inference tasks (token prediction)

Finding the pre-trained model that suits your needs

HuggingFace

- The ‘HuggingFace‘: Python library includes lots of datasets and transformer-based models
- ‘HuggingFace‘’s API works well with libraries such as ‘TensorFlow‘, ‘Keras‘, and ‘PyTorch‘.

Finding the pre-trained model that suits your needs

HuggingFace

- The ‘HuggingFace‘: Python library includes lots of datasets and transformer-based models
- ‘HuggingFace‘’s API works well with libraries such as ‘TensorFlow‘, ‘Keras‘, and ‘PyTorch‘.

Transfer learning paradigm

The idea of transfer learning is very powerful

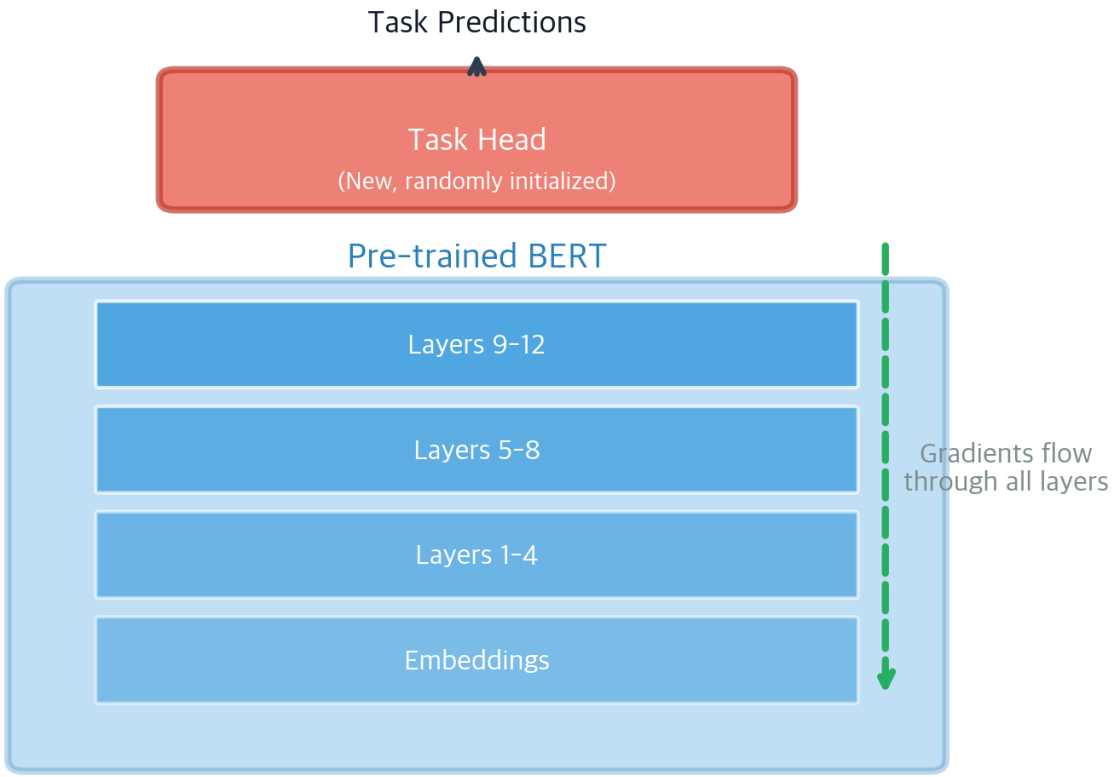
Transfer Learning paradigm

1. Pre-train a model on data that is at hand (e.g., Wikipedia, Google News)
2. Fine-tune the model on your downstream task (bring in your small-scale annotated dataset)

By adding 'task-specific' heads you can produce specific outputs, e.g., classification, text generation, named entity recognition, etc.

Fine-tuning BERT

Fine-tuning: End-to-End Training



Source:
https://mbrenndoerfer.com/writing/bert-finetuning-classification-ner-qa

Let's look at an example (exercises/transformers-custom-dataset.ipynb)

Transfer learning paradigm

Do I need all this fancy stuff?

Things to consider

How important is. . .

- precision/recall? Am I satisfied with .88 when .90 is theoretically possible? .85? .80? .75?
- explainability?
- computational resources?
- generalizability and out-of-sample performance?

Things to consider

How important is. . .

- precision/recall? Am I satisfied with .88 when .90 is theoretically possible? .85? .80? .75?
- explainability?
- computational resources?
- generalizability and out-of-sample performance?

Things to consider

How important is. . .

- precision/recall? Am I satisfied with .88 when .90 is theoretically possible? .85? .80? .75?
- explainability?
- computational resources?
- generalizability and out-of-sample performance?

Things to consider

How important is. . .

- precision/recall? Am I satisfied with .88 when .90 is theoretically possible? .85? .80? .75?
- explainability?
- computational resources?
- generalizability and out-of-sample performance?

Ethical considerations

(Ab-)using word embeddings to detect biases

Biased embeddings

- word embeddings are trained on large corpora
- As the task is to learn how to predict a word from its context (CBOW) or vice versa (skip-gram), biased texts produce biased embeddings
- If in the training corpus, the words "man" and "computer programmer" are used in the same context, then we will learn such a gender bias

Bolukbasi, T., Chang, K.-W., Zou, J., Saligrama, V., & Kalai, A. (2016). Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings, 1–25. Retrieved from http://arxiv.org/abs/1607.06520

Biased embeddings

Usually, we do not want that (and it has a huge potential for a shitstorm)

unless. . .

we actually want to chart such biases.

Biased embeddings

Usually, we do not want that (and it has a huge potential for a shitstorm)

unless. . .

we actually want to chart such biases.

Biased embeddings

Usually, we do not want that (and it has a huge potential for a shitstorm)

unless. . .

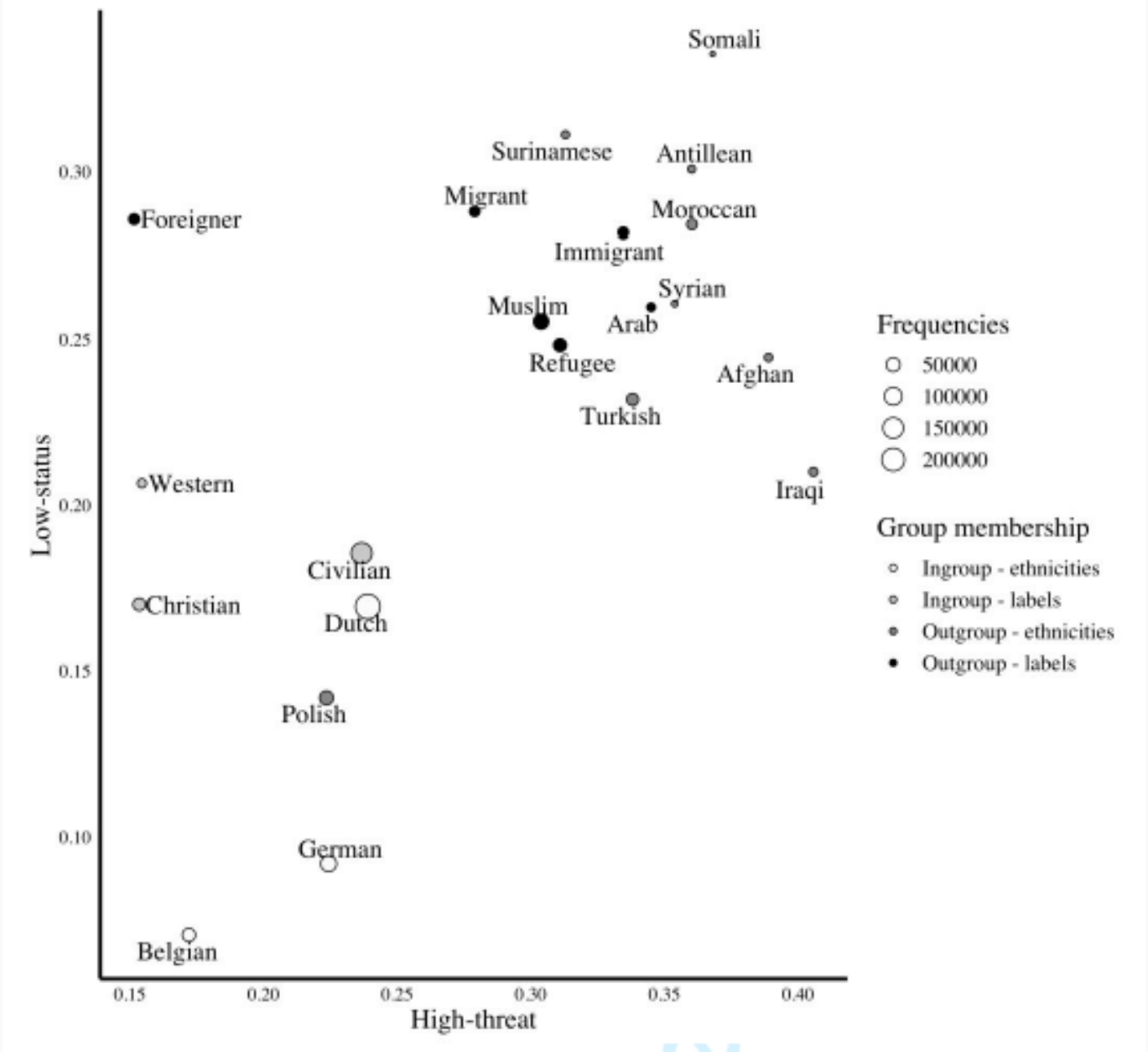
we actually want to chart such biases.

An example from our colleagues research

They trained word embeddings on 3.3 million Dutch news articles.

Are vector representations of outgroups (Maroccans, Muslims) closer to representations of negative stereotype words than ingroups?

Kroon, A.C., Van der Meer, G.L.A., Jonkman, J.G.F., &Trilling, D. (2021): Guilty by Association: Using Word Embeddings to Measure Ethnic Stereotypes in News Coverage. *Journalism & Mass Communication Quarterly*


Somali
Surinamese
Antillean
Migrant
Moroccan
Foreigner
Immigrant
Syrian
Muslim
Arab
Refugee
Afghan
Turkish
Iraqi
Western
Civilian
Christian
Dutch
Polish
German
Belgian
Low-status
High-threat
0.10
0.15
0.20
0.25
0.30
0.15
0.20
0.25
0.30
0.35
0.40
Frequencies
50000
100000
150000
200000
Group membership
Ingroup - ethnicities
Ingroup - labels
Outgroup - ethnicities
Outgroup - labels

Exercise

References

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). **BERT: Pre-training of deep bidirectional transformers for language understanding.** *NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*, *1*(Mlm), 4171–4186.